

## FFBank Application

FFBank is a web-based banking application that allows users to perform banking operations such as account management, money transfers, loan management, and cheque submissions. It also includes an admin interface for managing users, cheques, and loans. The application is built with a React frontend and a Node.js/Express backend, using MySQL as the database.

## Prerequisites

Before setting up the project, ensure you have the following installed: node.js, react, mysql, as given in package.json.

## Project Structure

```
bank-management-system/
├── uploads/
├── frontend/
│   ├── src/
│   │   ├── assets/
│   │   ├── components/
│   │   │   ├── UserLogin.js
│   │   │   ├── UserSignup.js
│   │   │   ├── UserDashboard.js
│   │   │   ├── TransferMoney.js
│   │   │   ├── UserSubmitCheque.js
│   │   │   ├── AdminLogin.js
│   │   │   └── ManageCheques.js
│   │   ├── App.js
│   │   └── index.css
├── bank_management_schema.sql
├── server.js
└── Functions.js
```

Setup the mysql

Import the mysql schema given or create two databases personal\_information and user\_credentials in personal\_information it store cheques, loan ,user\_info, transactions and in user\_credentials it store id, passwords of user ,admin.

### Start the backend server:

node server.js

The server will run at <http://localhost:3001>.

### Start the frontend development server:

npm start

The frontend will run at <http://localhost:3000>.

## Open the application

Go to <http://localhost:3000> in your browser.

Options for User Login, Admin Login, and Sign Up will be shown on the landing page.

## Using the Application

### User Features

#### Sign Up:

- Navigate to `/signup`.
- Enter an existing account number (from `user_info`) and a password.
- If the account exists and credentials don't already exist, the signup will succeed.

#### User Login:

- Go to `/user/login`.
- Enter your account number and password.
- Upon successful login, you'll be redirected to the user dashboard (`/user/dashboard`).

#### User Dashboard:

- **View Account Details** (`/user/account`): Displays account information.
- **Transfer Money** (`/user/transfer`): Transfer funds to another account.
- **Transaction History** (`/user/transactions`): View past transactions.
- **Loan Info** (`/user/loans`): View loans and pay EMIs.
- **Submit Cheque** (`/user/cheques`): Upload a cheque image and submit details.

### Admin Features

#### Admin Login:

- Go to `/admin/login`.
- Enter admin ID and password from `admin_credentials`.
- Upon successful login, you'll be redirected to the admin dashboard (`/admin/dashboard`).

#### Admin Dashboard:

- **Manage Users** (`/admin/users`): Fetch and update user information.
- **Manage Cheques** (`/admin/cheques`): Approve or reject pending cheques.
- **Manage Loans** (`/admin/loans`): View loans for a specific user.

## API Endpoints

### User Routes

- POST /user/login: Authenticate a user.
- POST /user/signup: Register a new user.
- GET /user/account/:account\_number: Get account details.
- POST /user/transfer: Transfer money.
- GET /user/transactions/:account\_number: Get transaction history.
- GET /user/loans/:account\_number: Get loan information.
- POST /user/emi/:loan\_id: Pay loan EMI.
- POST /submit-cheque: Submit a cheque (multipart/form-data).

### Admin Routes

- POST /admin/login: Authenticate an admin.
- GET /admin/cheques/pending: Get pending cheques.
- POST /admin/cheques/approve/:cheque\_id: Approve or reject a cheque.
- GET /admin/users/:user\_id: Get user details.
- PUT /admin/users/:user\_id: Update user details.
- GET /admin/loans/:user\_id: Get user loans.

### Notes

- Ensure the MySQL server is running before starting the backend.
- The cheque image upload feature requires the uploads/ directory to exist.
- Passwords are stored in plain text for simplicity; in a production environment, use a secure hashing library like bcrypt.
- The application uses Tailwind CSS for styling, included via CDN or local setup.

### Troubleshooting

- **Backend errors:** Check the console for database connection issues or missing dependencies.
- **Frontend errors:** Ensure the backend server is running and the API endpoints are accessible.
- **CORS issues:** The backend includes CORS middleware; verify it's configured correctly.
- **File upload issues:** Confirm the uploads/ directory has write permissions.

### **Commands to use database**

–To connect to mysql

```
mysql -u root -p
```

–to show Databases

```
SHOW DATABASES;
```

–to connect to certain database

```
USE personal_information
```

```
USE user_credentials
```

-- to View user information

```
SELECT * FROM user_info;
```

-- View User Login Credentials

```
SELECT * FROM user_credentials;
```

-- View Admin Credentials

```
SELECT * FROM admin_credentials;
```

-- View Transaction History

```
SELECT * FROM transactions;
```

-- View Loan Details

```
SELECT * FROM loans;
```

-- View Cheque Details

```
SELECT * FROM cheques;
```

### **Commands to run backend**

```
node server.js
```

### **Commands to run frontend**

```
npm start
```

